



QUEUEING SIMULATION SYSTEM

Data Structure & Algorithms

SPRING 2026

TEAM MEMBERS:

Mazen Thobian	320250329
Sameh Waleed	320250314
Anass Elsakka	320250317
Zain elden Basil	320250319
Sama Fady	320250311
Karen Maged	320250326
Omar Khaled	320250299

GROUP 4, SECTION 12

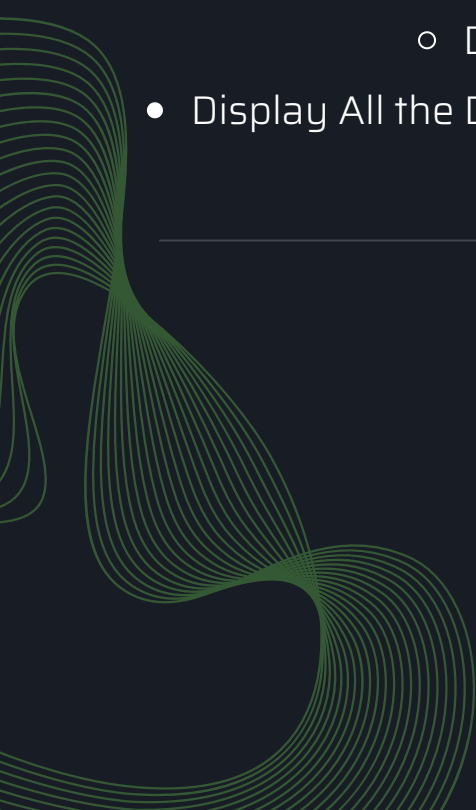


QUEUEING SYSTEM

Is a model that describes how customers wait in line and get served.

OBJECTIVES

- Simulate:
 - Customers arrival.
 - Service.
 - Measure:
 - Waiting time.
 - Queue length.
 - Analyze performance with:
 - Different server counts.
 - Different simulation times.
 - Display All the Data in a UI.
-



DATA STRUCTURE USED

QUEUE (FIFO)

FIFO stands for First In, First Out, and it's the rule the queue follows.

The Queue Class (Header File):

```
class Queue {  
    private:  
        int* arr;  
        int capacity;  
        int front;  
        int rear;  
        int size;  
  
        void resize();  
  
    public:  
        Queue();  
  
        void enqueue(int x);  
  
        void dequeue();  
  
        int peek() const;  
  
        bool isEmpty() const;  
  
        int getSize() const;  
  
        void display() const;  
};
```

The Queue (.cpp File)

Constructor **Queue()** :

```
Queue() {  
    capacity = 10;  
    arr = new int[capacity];  
    front = 0;  
    rear = -1;  
    size = 0;  
}
```

Method, **resize()** :

```
void resize() {  
    int newCapacity = capacity * 2;  
    int* newArr = new int[newCapacity];  
  
    for (int i = 0; i < size; i++) {  
        newArr[i] = arr[(front + i) % capacity];  
    }  
  
    delete[] arr;  
    arr = newArr;  
    capacity = newCapacity;  
    front = 0;  
    rear = size - 1;  
}
```

The Queue (.cpp File) cont.

Method, **enqueue()** :

```
void enqueue(int x) {  
    if (size == capacity) resize();  
    rear = (rear + 1) % capacity;  
    arr[rear] = x;  
    size++;  
}
```

Method, **dequeue()** :

```
void dequeue() {  
    if (isEmpty()) {  
        cout << "Queue is Empty\n";  
        return;  
    }  
    front = (front + 1) % capacity;  
    size--;  
}
```

The Queue (.cpp File) cont.

Method, **peek()** **const** :

```
int peek() const {  
    if (isEmpty()) {  
        cout << "Queue is Empty" << endl;  
        return -1;  
    }  
    return arr[front];  
}
```

Method, **isEmpty()** **const** :

```
bool isEmpty() const {  
    return size == 0;  
}
```

The Queue (.cpp File) cont.

Method, **getSize() const** :



```
int getSize() const {  
    return size;  
}
```

Method, **display() const** :



```
void display() const {  
    if (isEmpty()) {  
        cout << "Queue is Empty" << endl;  
        return;  
    }  
    cout << "Queue: ";  
    for (int i = 0; i < size; i++) {  
        cout << arr[(front + i) % capacity] << " ";  
    }  
    cout << endl;  
}
```


HOW THE APP IS BUILT

C++

1. The **Queue** Class.
2. Handling all the Data Structure Logic.

Queue.h / Queue.cpp

C++/CLI (Managed C++)

To Call **C++** inside .NET Environment.

HomePage.h (**Form**), The GUI

.NET Framework (WinForms)

Provides the GUI ToolKits
(**Buttons**, **TextBoxes**, **Timers**, **CheckBoxes**,
ProgressBars, etc)

HomePage.cpp ,Entry Point That Runs
the Form using **.NET Framework**

HOW THE SIMULATION WORKS ?!

When You Press (**Start**):

- Everything Resets (Time, Statistics, IDs).
 - The user chooses both:
 - The Number of Servers from the Combo Box.
 - The Simulation Time from another Combo Box.
 - Timer starts ticking every **1 Second**.
-

What Happens Every Tick ?

Step 1: Ticking down busy servers

each server that is busy, subtract 1 from its remaining service time and when it reaches 0, that server is free again.

Step 2: Auto Arrival (if enabled by Check Box)

`rand() % 3 == 0`

means 33% chance that a customer arrives

if yes, assign them an ID, record their arrival time, give them a random service time (1-4 seconds), add them to the queue.

Step 3: Assigning free servers

loop through all servers

if a server is free (busy = 0) AND queue is not empty:

- Take the first customer from the queue (FIFO)
- Calculate their **wait time = currentTime - arrivalTime**
- Set that server busy for their service time
- Count them as served

Step 4: Updating the UI

- show which server served which customer.
- show current queue length, total served, total arrived.
- Update progress bar.

Step 5: Checking if Done

- If it's manual mode (Check Box isn't checked and the Enqueue Button is used) and the queue is empty: **stop**.
- If time ran out: **stop** and show final stats.

When the Simulation End:

- User is going to see :
 - Number of Total Served Customers.
 - Number of Total Arrived Customers.
 - Average Waiting Time =

Total Time / Number of Total Served Customers

How to easily understand it :

- **Queue:** is like Customers in line waiting.
 - **Servers:** are like Employees.
 - **Each Tick:** is 1 Second passing.
-

FEATURES

1. Auto / Manual Arrival Mode.
 2. Validation for Wrong Actions.
 3. Selectable Number of Servers. (1, 2, 3)
 4. Selectable Simulation Time. (30, 60, 120)
 5. Generating IDs for Customers.
 6. UI for Controlling & Displaying the Simulation.
 7. Progress Bar.
 8. Real Time Statistics:
 - a. Queue Length.
 - b. Number of Arrived Customers.
 - c. Number of Served Customers.
 9. Auto Simulation-Ending When all Customers are Served in the Manual Mode.
 10. Average Waiting Time.
 11. Reset Button.
-

RESULTS & ANALYSIS

1.Auto Simulation

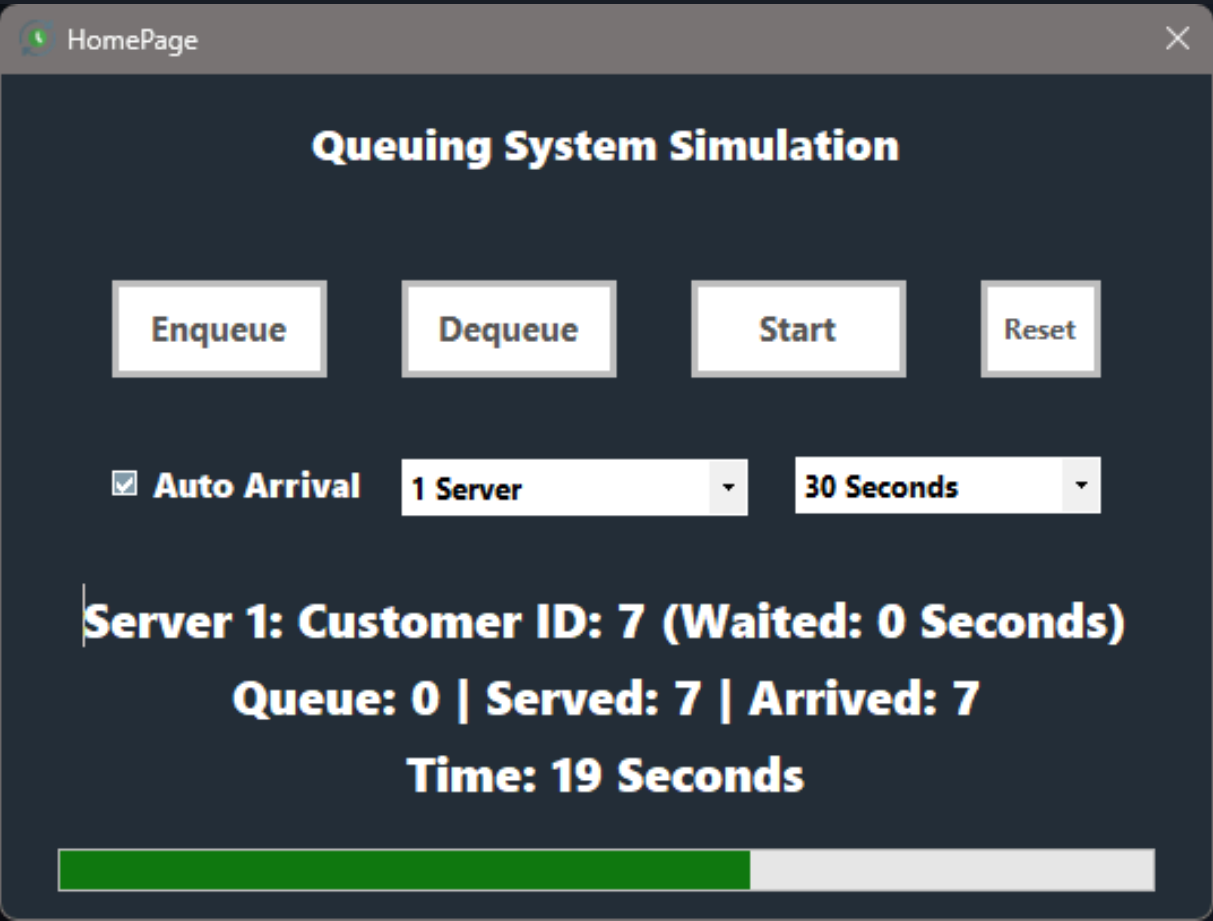
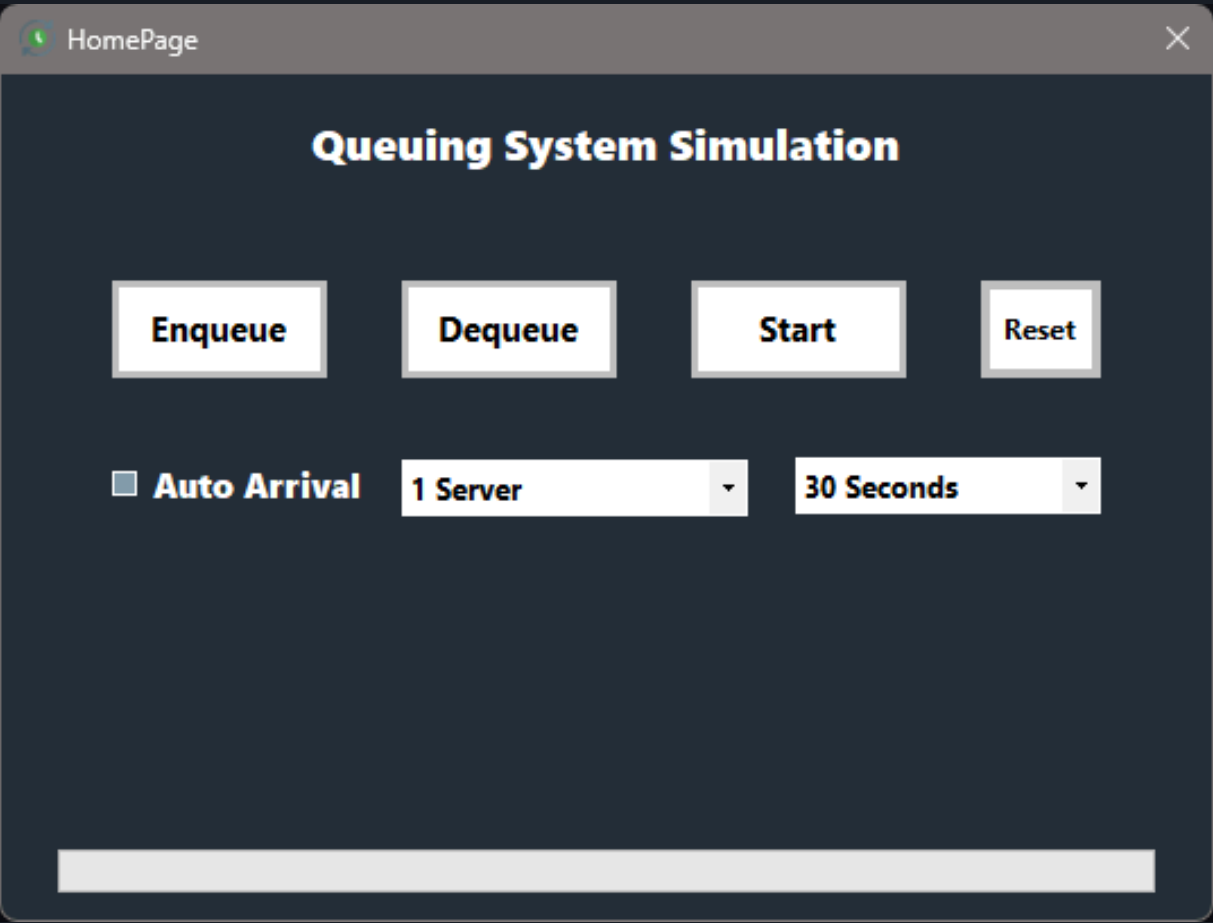
Servers	Arrived Customers	Served Customers	Average Time
1	8	8	0
2	10	10	0
3	13	13	0

2.Manual Simulation: 10 Customers

Ends Simulation When all Customers are Served

Servers	Served Customers	Average Time
1	10	12
2	10	4
3	10	3

SCREENSHOTS



SCREENSHOTS CONT.

HomePage

×

Queuing System Simulation

Enqueue

Dequeue

Start

Reset

☒ Auto Arrival

1 Server

30 Seconds

Done ! Served: 9 | Arrived: 10

Avg wait: 2 Seconds

Simulation Completed !





```
#include <iostream>
using namespace std;

int main(){

    cout << "Thank You !" << endl;

}
```